

Brooklyn Nets Data Scientist Basketball Project

Ruslan Davtian

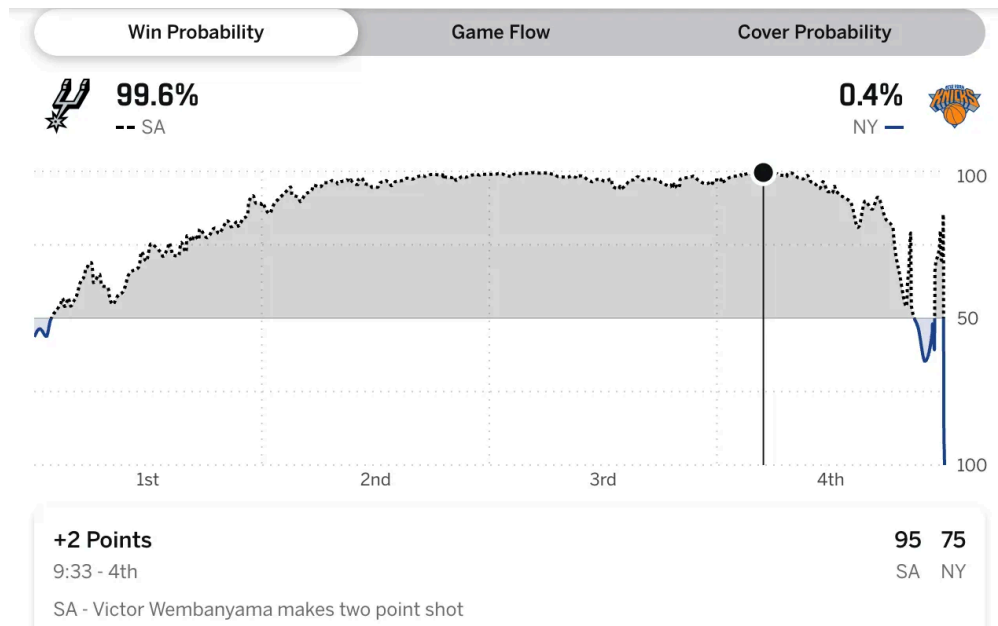


Toggle code visibility

Building an NBA Win Probability Model for Front Office Decision-Making

1. Background and Research Question

Front offices and coaching staff routinely rely on intuition and experience when evaluating "clutch" performance, late-game strategy, and in-game decision-making. A calibrated, play-by-play win probability model turns that intuition into something measurable: at any point in a game, given the score, time remaining, and pregame context, what is the probability the home team wins? These models exist publicly such as ESPN's live win probability which saw some crazy line changes in this year's playoffs for the New York Knicks such as the one below, a 29-point 2nd half comeback in the finals.



Once that model exists, the change in win probability from one play to the next — **win probability added (WPA)** — becomes a metric for crediting individual plays and players with how much they actually moved a team toward winning, independent of raw counting stats. This allows us to understand why such improbable comebacks happen and which players or plays contribute most to seeing a 0.4% win percentage move to 100%.

This project set out to answer a layered set of questions:

- Can a reliable, calibrated win probability model be built from public play-by-play data?
- Once built, does it agree with an established external benchmark (ESPN's own live/pregame win probability)?
- Can WPA, derived from that model, surface useful signal about player and team value — especially in the 4th quarter or clutch situations that front offices care most about?

The rest of this report documents the full pipeline built to answer these questions, in the order it was actually built: data collection, feature engineering, missing-value handling, model construction, model evaluation, and finally the WPA analysis layer itself.

2. Data Collection

I utilized the publicly available sportsdataverse package

(<https://py.sportsdataverse.org/docs/nba/>) to easily use its functionality to scrape NBA play-by-play, team schedule, team boxscore, and ESPN pregame betting lines and game-level metadata data. These raw data sets can be filled with missing or bad data and a big advantage of using this package is that it already cleans and validates data for completeness which can save time from performing data validation techniques. I used two full seasons (2025, 2026) for the working model, roughly 600,000 play-level rows

per season (~500 rows/game across ~1,230 regular-season games plus postseason excluding all-star games). Each row tells us what happened in that possession, who was the main player on that play, how much time remaining in the game, which team possessed the ball, etc. Below is a scrollable example of 3 rows of play-by-play data to get a sense of the information available.

DF Shape: 1249831 rows and 69 cols

id	pregame_point_diff_avg	home_team_won	pregame_spread_home_team	home_team_won
6	-5.545455	0	-15.5	
6	-5.545455	0	-15.5	
6	-5.545455	0	-15.5	

3. Feature Engineering

In this modeling problem, we need to define the target variable we want to forecast (did the home team win or lose) as well as any features or information that we know before the next possession occurs that can help predict which team wins. The target variable is defined as a binary (1/0) feature based solely on the final score, `home_team_won`. This is the single most important modeling decision in the whole build: a row showing the home team down 15 late in the 3rd quarter still carries the label of whatever the game's *final* outcome was, not the momentary state. Without this, the model would have no way to learn what a blown lead or a genuine comeback looks like in the underlying features.

When modeling in-game play by play data, we want to be cautious about the information we use in the model to make sure that when making forecasts for future games, they must be known before the upcoming play finishes. This is known as forward leaking and if future information leaks into the model, we may have a model that performs very well in training, but is much worse on new unseen data. Therefore, I made an effort to create 1-play lagged features for any in-game context or in-game rolling stats.

Feature	Description
<code>score_diff_lag</code>	Home – Away score margin, lagged by one play so a row reflects the score before that row's play resolves, not after. Skipping this lag would let the model see the outcome of the very shot it's supposed to be predicting the value of.
<code>start_game_seconds_remaining</code>	Time remaining at the start of the possession, computed per-period with explicit overtime handling
<code>score_diff_lag_time_remaining_interaction</code>	$\frac{\text{score_diff_lag}}{\sqrt{\text{seconds_left_in_game} + 30}}$ — the core engineered interaction, capturing that the same score margin is far more decisive with little time left than with the whole game ahead
<code>period_number</code>	1–4, with 5+ collapsed into a single "OT" bucket
<code>is_home</code>	1 if the team about to possess the ball is the home team — a genuine possession indicator that flickers row to row, distinct from a static "home team" flag
<code>is_playoffs</code>	Regular season vs. playoffs context flag
<code>home_timeouts_remaining_lag</code>	Cumulative timeout usage counted <i>before</i> the current row's own event, with the correct NBA allotment logic (7 per team in regulation, +2 per side per overtime period)
<code>away_timeouts_remaining_lag</code>	Cumulative timeout usage counted <i>before</i> the current row's own event, with the correct NBA allotment logic (7 per team in regulation, +2 per side per overtime period)
<code>timeouts_diff_lag</code>	Home - Away timeouts lagged remaining
<code>timeout_diff_lag_time_remaining_interaction</code>	Similar interaction term to score diff & seconds remaining to capture the added complexity of having multiple timeouts remaining late in the game versus early in the game

Feature	Description
<code>away_back_to_back</code>	Whether or not the away team played the day before
<code>home_back_to_back</code>	Whether or not the home team played the day before
<code>rest_days_diff</code>	Home rest days - Away rest days
<code>pregame_point_diff_avg</code>	Static pregame prior of the possession team's average score margin
<code>pregame_w_pct</code>	Static pregame prior of the possession team's winning percentage leading up to the game
<code>pregame_spread_home_team</code>	ESPN's closing pregame home team spread line (provided by ESPN Bet or DraftKings)
<code>pregame_home_team_odds_favorite</code>	Binary (1/0) whether or not home team is the pregame favorite
<code>pregame_home_team_w_prob</code>	ESPN's closing pregame home team win probability estimate converted from moneyline odds to probability

I decided to include some of these pregame static priors to adjust for team strength and pregame winning expectation given that matchups in the NBA can be lopsided given Thunder/Spurs regular season dominance. This also could give more credit towards players on weaker teams that make contributions to winning than those same plays for players on stronger teams. Given more time, I would investigate with additional in-play features focused on team scoring runs, consecutive defensive stops, personal fouls/bonus situations, and in-game shooting percentages to better contextualize team and player performance.

4. Missing Value and Feature Analysis

Tree-based models such as XGBoost handle missing values natively, so the modeling pipeline does not impute manually. Instead, missingness is surfaced and monitored rather than imputed. Nevertheless, it is always important to do a distribution check of each feature and know where missing values exist. Below is a frequency count as well as proportion of rows missing for each feature that has a missing value. We see that metadata such as years of experience in NBA, player_name, position, and age have the most values missing. None of these columns are features in the model and those missing rows must correspond to plays without a player such as team timeout. A low percentage of total values is missing from the actual features used in the model related to pregame home team win probability. Since the model itself is able to learn from rows with missing

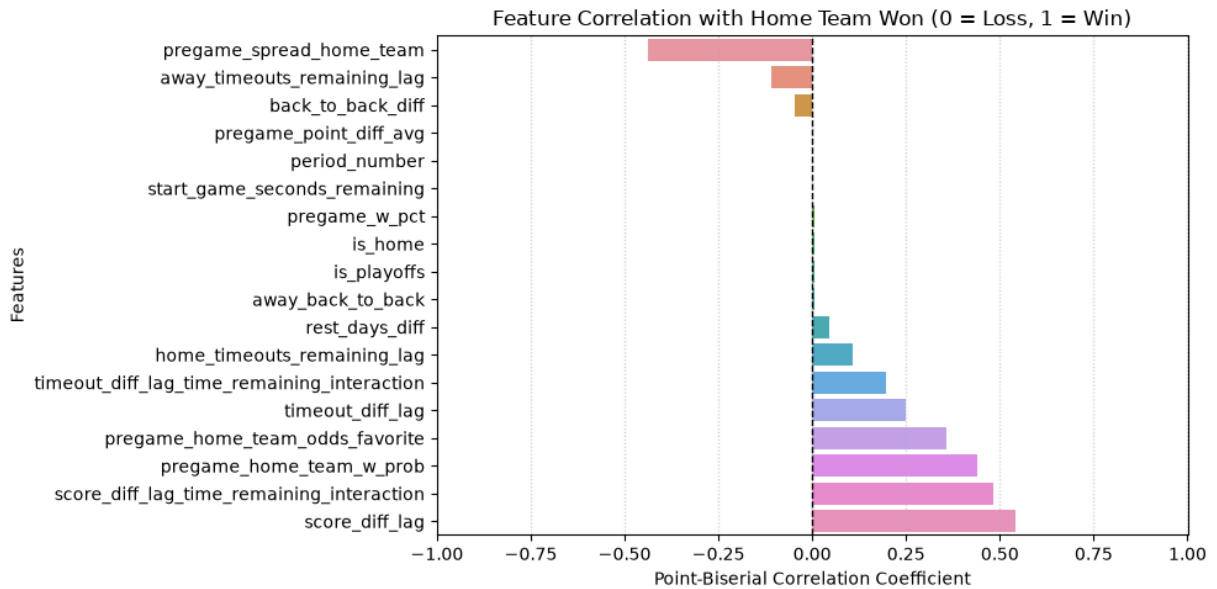
data and the missing count is less than 1%, I decided to leave the data set as-is and not eliminate any rows or impute missing data.

Your selected dataframe has 69 columns.
There are 11 columns that have missing values.

Out[31]:

	Missing Values	% of Total Values
experience	141177	11.3
player_name	141131	11.3
position	141131	11.3
age	141131	11.3
athlete_id_1	84106	6.7
pregame_w	15104	1.2
pregame_l	15104	1.2
away_team_odds_money_line	2540	0.2
home_team_odds_money_line	2540	0.2
pregame_home_team_w_prob	2540	0.2
espn_home_team_win_prob	36	0.0

This chart below checks which pregame and situational features carry the strongest linear relationship with the eventual outcome before any modeling begins. Given the feature set includes pregame_home_team_w_prob, pregame_spread_home_team, and pregame_home_team_odds_favorite alongside situational features like timeout_diff_lag and rest_days_diff, this is the first checkpoint for whether the market-derived pregame signals are actually the dominant predictors before the model has a chance to learn anything. The features with the highest correlation or signal to predict home team victory are pregame spread/moneyline odds, in-game score differential, and the interaction between score differential and time remaining in the game.



5. Model Building

Due to time constraints, I limited my choice of machine learning algorithms to just XGBoost without comparing to any other binary classification model. This model has been the industry standard for some time as having strong predictive power as it is able to discover complex feature interactions and handle mixed continuous/binary/categorical features without heavy preprocessing as well as missing values.

Since each season has plenty of in-game play-by-play data, I decided to use all of 2025 (reg season + playoffs) as the train set and 2026 as the holdout validation set to make predictions on. It is important to call out that each row is not fully independent as plays within a single game are correlated. Therefore, a random k fold cross validation (CV) setup does not suffice, so I utilized grouped 5-fold cross validation instead grouped on game_id. Cross validation is applied within the train set where 4/5th of the train data is split for training and the remaining 1/5 used for testing. The model is fit five times until each datapoint has been used for testing. The model that minimizes average log loss across the 5 test splits will be the final, best model. This grouped 5-fold CV approach ensures that no single game's rows ever appear on both sides of a split during cross validation. I used a manual grid search for hyper-parameter tuning to find the best parameters that minimized log loss. After fitting the model on the training data and finding the best parameters that minimized logloss through cross validation of the train set, the best parameters model was used to make predictions on the 2026 validation set. Isotonic calibration and monotonic constraints on certain features were both part of an earlier, more rigorous version of this pipeline, then explicitly removed to produce a simpler baseline to start from.

Below are performance metrics (accuracy, recall, precision, f1 score, log loss, brier score) resulting from predictions on the test set. I also compute the same error using ESPN's own live in-game win probability model as a check to make sure model leakage is

not an issue and calibration to be similar to that of ESPN. I believe using ESPN's pregame odds in the model itself is the main reason why performance is similar as both models would start in a similar pregame state. We also see the confusion table printed looking at frequency and proportion of games from a binary win/loss perspective using 0.5 as the decision threshold for home team winning.

```
Train X's Dim: (616159, 18)
Train Y Dim: (616159,)
Test X's Dim: (633672, 18)
Test Y Dim: (633672,)
Total Rows: 1249831
Fitting 5 folds for each of 48 candidates, totalling 240 fits
Best CV logloss: 0.4523
Best params: {'learning_rate': 0.005, 'max_depth': 3, 'n_estimators': 900}
```

```
=== Test set performance ===
```

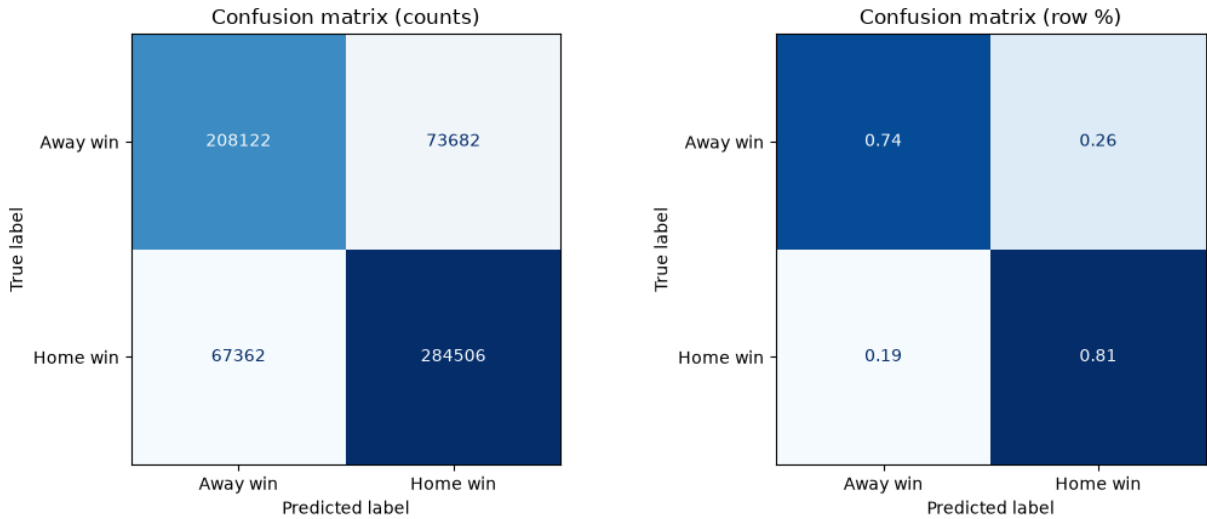
```
accuracy: 0.7774
precision: 0.7943
recall: 0.8086
f1: 0.8014
logloss: 0.4450
brier: 0.1476
```

```
=== ESPN Win Prob Test set performance ===
```

```
accuracy: 0.7766
precision: 0.7964
recall: 0.8029
f1: 0.7996
logloss: 0.4434
brier: 0.1480
```

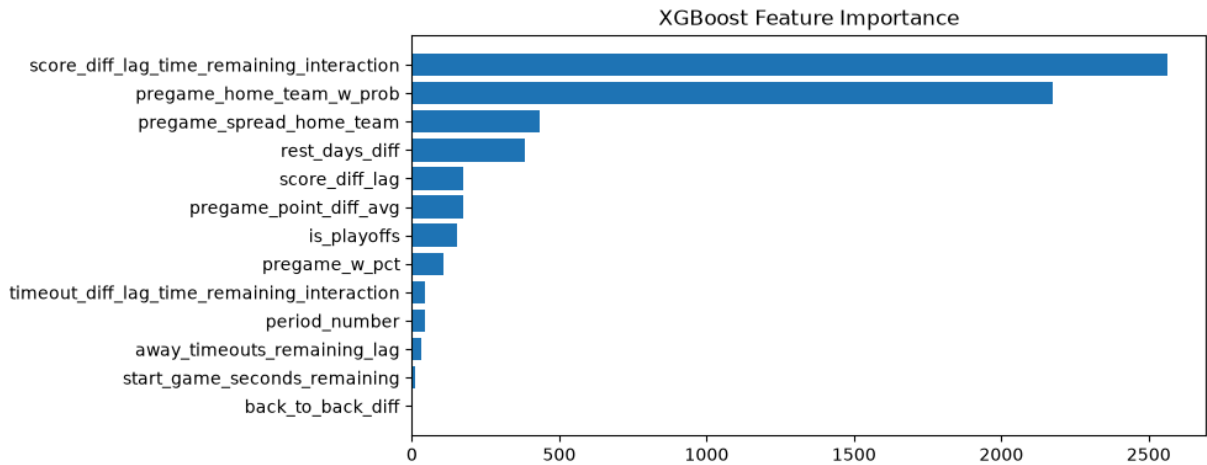
```
=== Classification report (threshold = 0.50) ===
```

	precision	recall	f1-score	support
Away win	0.76	0.74	0.75	281804
Home win	0.79	0.81	0.80	351868
accuracy			0.78	633672
macro avg	0.77	0.77	0.77	633672
weighted avg	0.78	0.78	0.78	633672



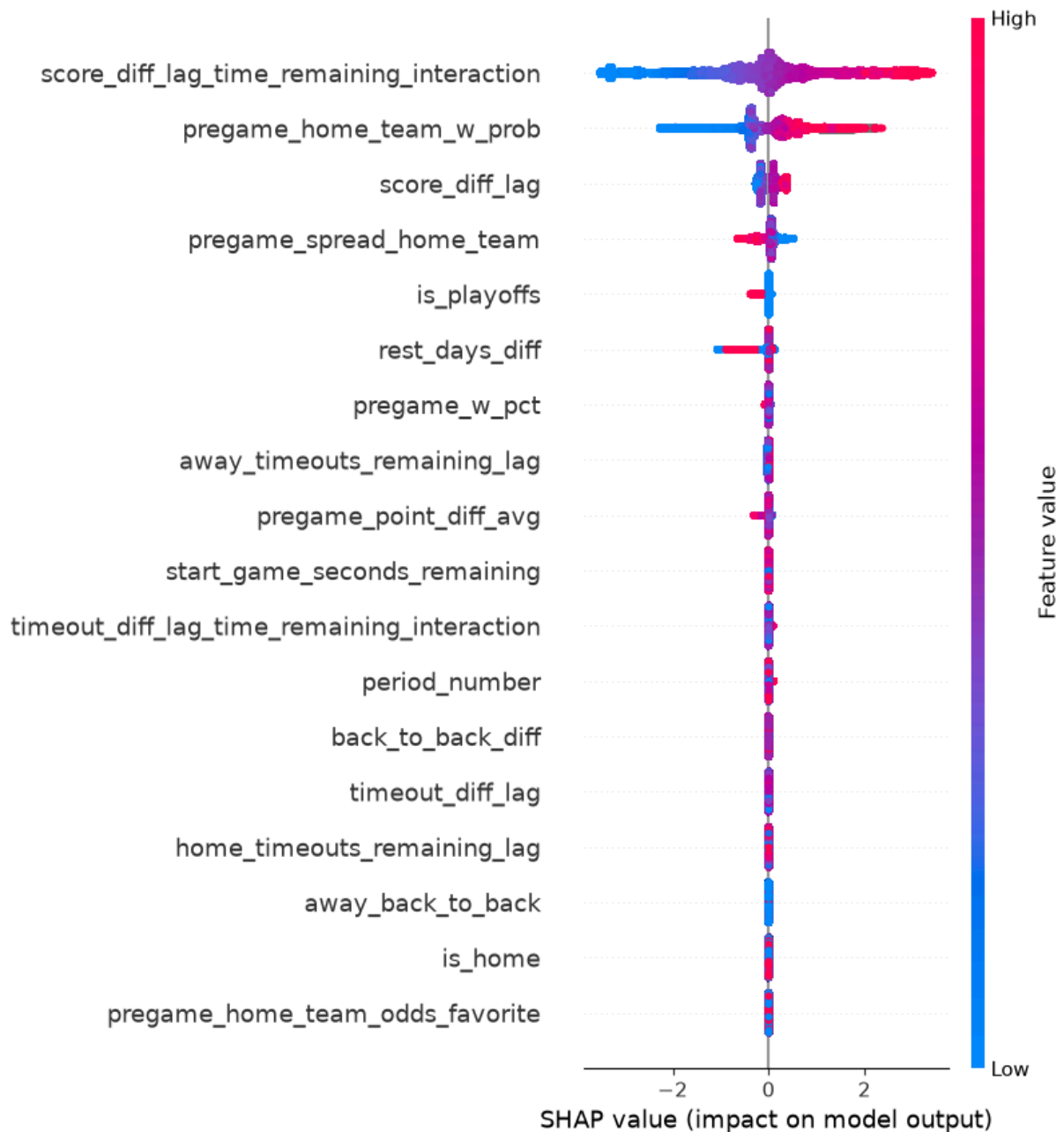
6. Analyzing Model Output

This plot below is a model feature importance plot which ranks features the model actually leaned on most heavily when constructing its decision splits. The expectation, per basketball logic, is that score difference and time remaining in the game should dominate. We see that the top two most important features by far are the interaction between score differential lagged and time remaining as well as the pregame home team win probability converted from ESPN moneyline odds. We also see pregame spread as well as the team difference in rest days as important features. This confirms the model isn't purely reactive to only in-game state — it's carrying forward real pregame information about relative team strength, rest status, regular season vs playoffs indicator throughout the entire game, not just at tipoff.



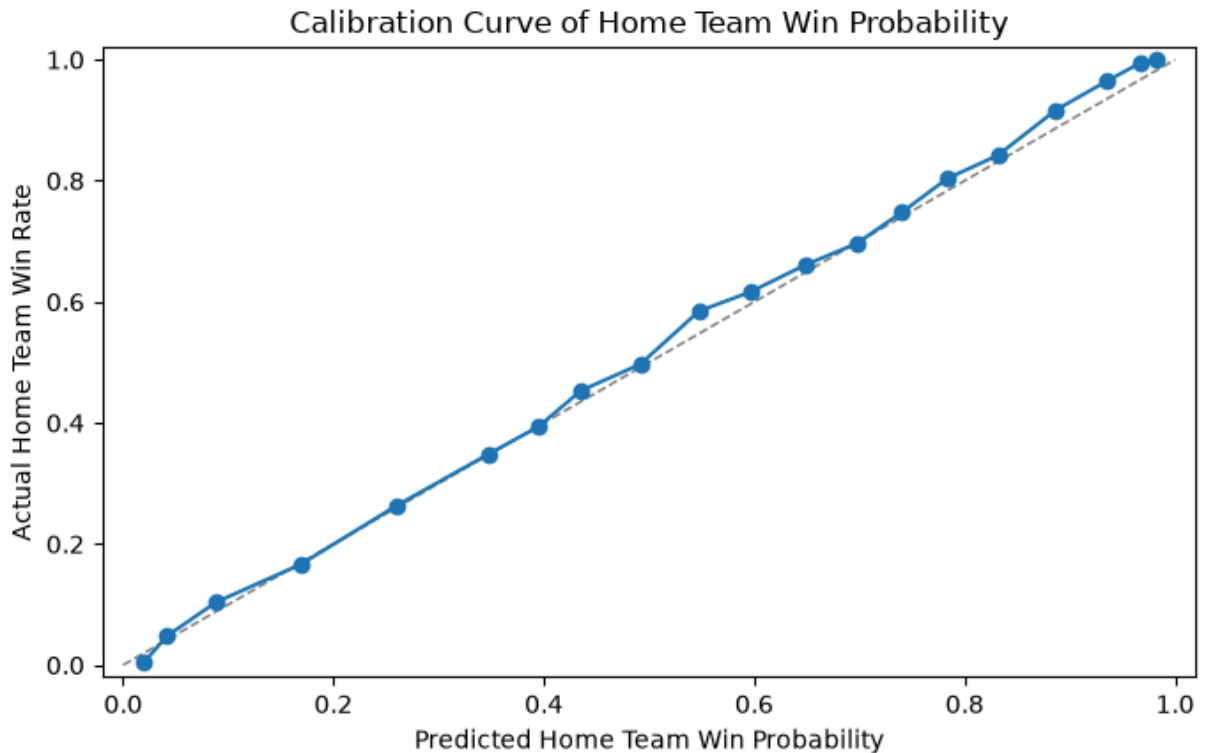
This complements the gain-based importance chart with directional detail: for the top-ranked features, do high feature values (red) push predictions toward a home win (positive SHAP, right side) and low values (blue) push toward an away win (left side). Each dot on this plot represents one possession or prediction row in the test set and we can see the top feature (score_diff_lag_time_remaining_interaction) has the most influence or impact in making model predictions where higher values push toward higher

home team win probabilities and lower values push toward lower home team win probabilities. This is consistent with the ~ 0.5 correlation coefficient we observed against the target variable. A similar pattern exists for pregame home team win probability and even spread (reverse direction). Interestingly, there seems to be more of an effect on away team win probability if the away team has more rest days than the home team (negative rest_days_diff). Positive rest days diff where home has more rest does not seem to impact their win probability much at all, most likely because they already have home court advantage to begin with.

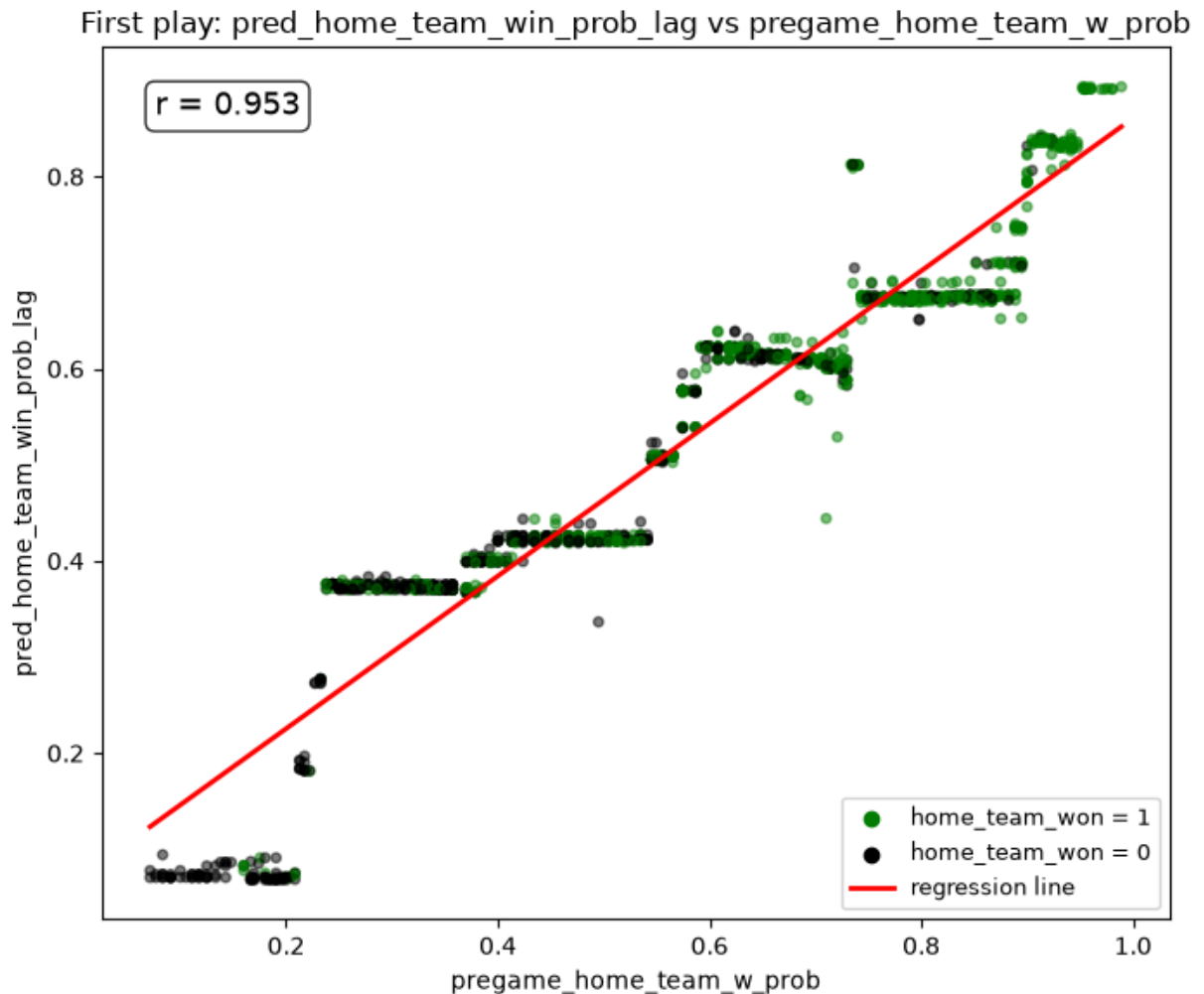


In terms of model calibration, it is important to check if a 70% model prediction generates the correct prediction 70% of the time which is what a well-calibrated model achieves. Plotted across 20 probability bins, this is the direct visual test of whether the model's stated confidence matches reality — a well-calibrated model's curve should sit

close to the 45-degree reference line across the full range. In our case, the model does sit along the diagonal line with some slight under-prediction around 50% and 85-90% win probability.



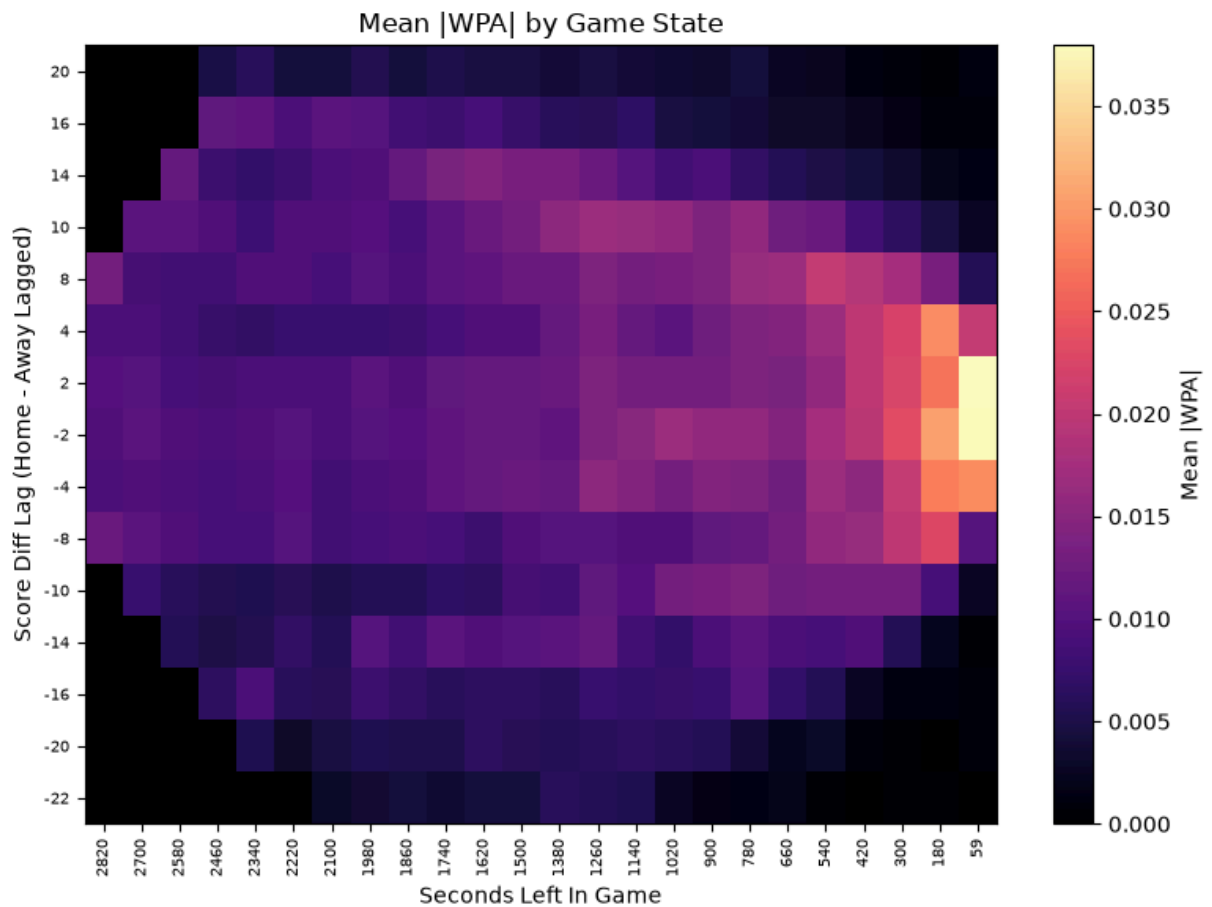
Lastly, I was curious to visualize how much of the pregame odds for home team winning influenced the starting prediction point for the game. Without any adjustments to matchup strength, I would expect home team win probability at the start of each game to hover closer to 50/50. But if we look at the scatterplot below isolating each game's first possession and comparing model predictions against the pregame odds from ESPN, the correlation coefficient is very strong, confirming the pregame features are doing real work from the opening tipoff. It's also color-coded so we can see the many high pregame/first play probabilities for home team did result in a home team victory and vice versa for the away team as green dots dominate top right and black dots dominate bottom left.



7. Using the Model for WPA Analysis on 2026 Validation Set

Now that we have a win probability, we can analyze win probability added (WPA) which is simply the win probability immediately after a possession minus the win probability immediately before it. The difference represents the change in win probability as a result of the play. We expect the magnitude of in WPA to be smaller early in games and larger late in games, especially the 4th quarter. And any changes in score differential such as a 3pt play should have the largest effect on win probability as well as scoring in general. Let's take a look at the magnitude or absolute value of WPA across different states in the game (lagged score differential interacted with seconds left in the game at possession start).

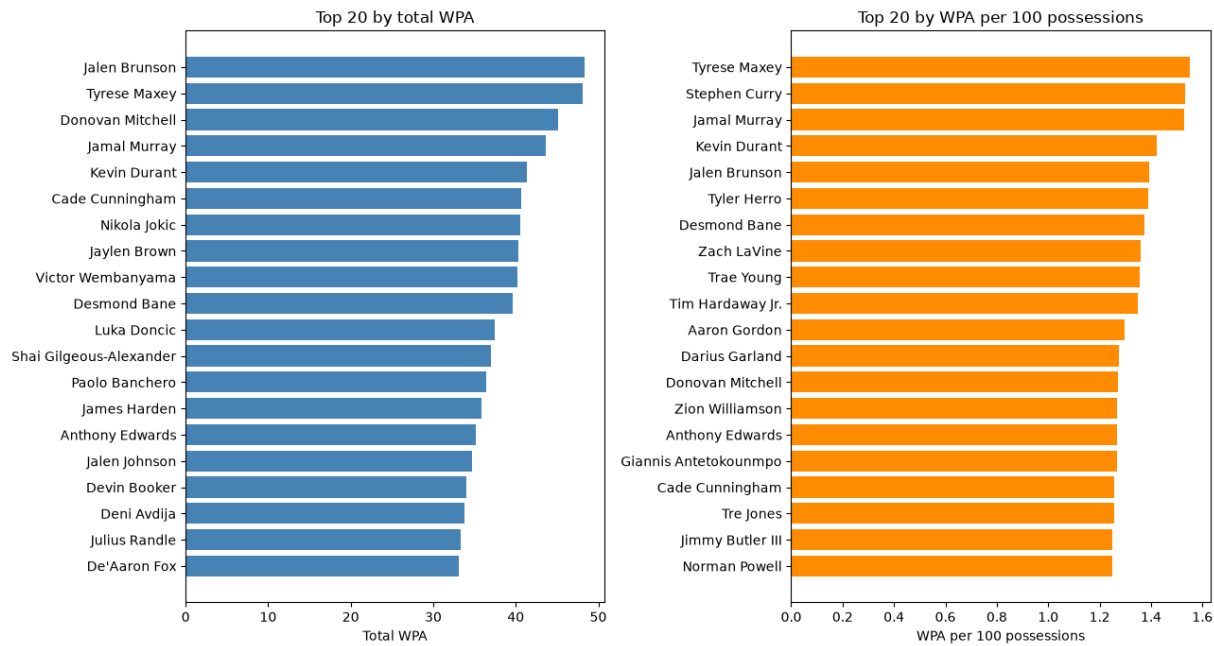
This heatmap shows where in the state space — combinations of score margin lagged and time remaining in the game — a single play carries the most weight toward the final outcome. The expected pattern is a concentration of high mean $|WPA|$ in the corner where time is short and the margin is close, fading to near-zero everywhere else (early game, or any point where the margin is already decisive). This is the direct visual confirmation that the model's sense of "leverage" matches actual basketball intuition about when moments matter most.



In terms of player ranking on WPA, we can aggregate in two different ways...

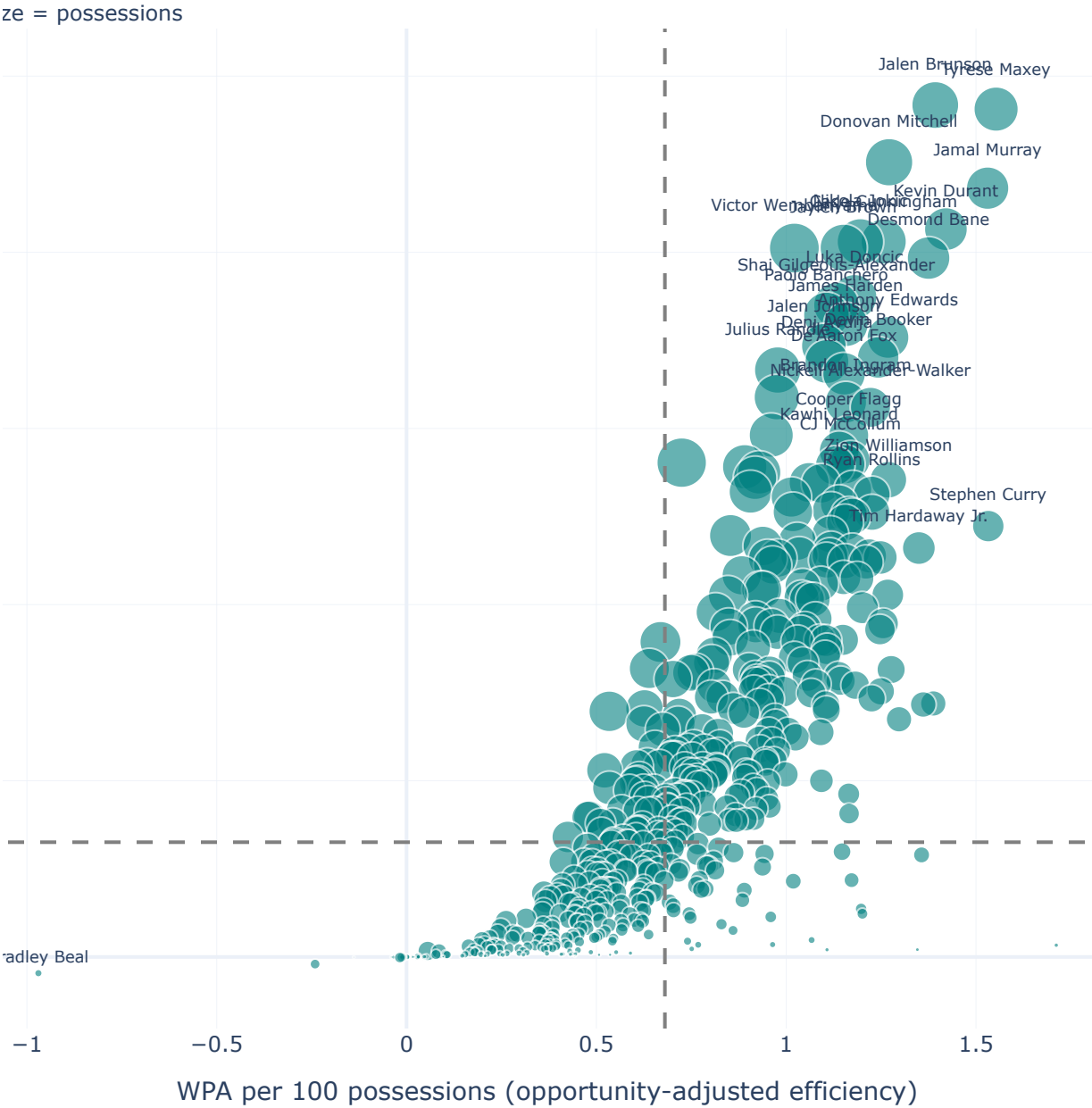
- **Total WPA** — cumulative, *not* adjusted for opportunity; naturally larger for players who played more possessions. Answers "how much real winning value did this player's actual role produce."
- **WPA per 100 possessions** — opportunity-adjusted; answers "how effective is this player independent of how much they played."

These intentionally answer different questions and are visualized together: a leaderboard comparing top-20 by each, and an interactive quadrant scatter plot (rate vs. total, bubble size = possessions) that separates "established, high-volume value" from "efficient but underused" from "propped up by usage" players.



In the plot below, the dotted black lines that create the 4 quadrants are the median values for total WPA and WPA per 100 possessions. The plot has an exponential growth shape to it as the top right set of players represent the elite (high efficiency and high volume of +WPA). Moving down from the top-right quadrant, the bottom right group depicts high efficiency players but a bit underrated due to lack of possession or scoring opportunities. Any player below the medians most likely did not have enough opportunities to make a significant impact on cumulating positive WPA.

Value Quadrant: Efficiency vs. Total Contribution

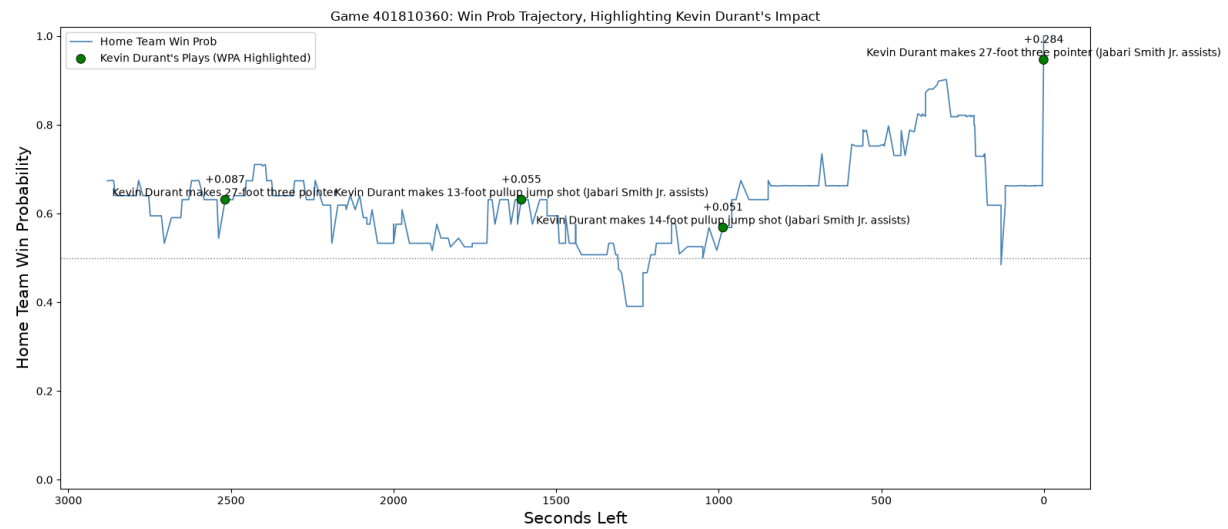


Let's also view WPA top rankings when filtering for 4th quarter and OT only to observe who was most clutch this past season (min. 300 possessions). In terms of efficiency on a per possession opportunity basis as the main consumer of the play, we see Kevin Durant had the highest WPA per possession adjusted for playing time in the league. This list checks out seeing some of the highest scorers in the league and best players among the top 20 (KD, SGA, Maxey, Curry, Jokic, Brunson, Ant Man, Giannis, etc.)

Out [18]:

	player_name	position	wpa_total	possessions	wpa_per_100
300	Kevin Durant	F	14.84	609.00	2.44
503	Tyrese Maxey	G	18.47	775.00	2.38
485	Tre Jones	G	8.18	344.00	2.38
214	Jamal Murray	G	18.10	764.00	2.37
306	Keyonte George	G	10.19	439.00	2.32
454	Shai Gilgeous-Alexander	G	13.88	603.00	2.30
460	Stephen Curry	G	8.77	398.00	2.20
108	Darius Garland	G	6.87	319.00	2.15
384	Nikola Jokic	C	15.55	723.00	2.15
328	Lauri Markkanen	F	7.59	363.00	2.09
127	Desmond Bane	G	15.89	804.00	1.98
203	Jalen Brunson	G	15.13	766.00	1.98
442	Ryan Rollins	G	10.08	522.00	1.93
29	Anthony Edwards	G	11.96	620.00	1.93
387	Nikola Vucevic	C	9.27	487.00	1.90
164	Giannis Antetokounmpo	F	7.91	421.00	1.88
238	Jerami Grant	F	8.27	441.00	1.88
65	Cade Cunningham	G	15.46	831.00	1.86
350	Matas Buzelis	F	9.72	523.00	1.86
443	Saddiq Bey	G	10.32	572.00	1.80

Analyzing a specific Kevin Durant game, this plot below traces one specific game's full win-probability curve and highlights the possessions where Kevin Durant's actions produced the top 4 largest swings. In this case, the top 4 |WPA| all resulted in positive win probability swings for his team, with his 3-point game winner vs Suns with 1 second left adding nearly 30% win probability to ice the game. Also printed is the corresponding dataset rows for these four plays along with a clickable link to the game-winning shot.



DF Shape: 4 rows and 12 cols

id	clock_display_value	home_score	away_score	text	short_description	pred_hon
4	0:1.1	100	97	Kevin Durant makes 27-foot three pointer (Jabari Smith Jr. assists)	+3 Points	
1	5:58	12	16	Kevin Durant makes 27-foot three pointer	+3 Points	
2	2:47	50	53	Kevin Durant makes 13-foot pullup jump shot (Jabari Smith Jr. assists)	+2 Points	
3	4:27	72	76	Kevin Durant makes 14-foot pullup jump shot (Jabari Smith Jr. assists)	+2 Points	

Out [22] :



Diving into the 2025-2026 Brooklyn Nets roster along with Julius Randle (new acquisition), we can see how Nets players stack up amongst each other in WPA rankings. Julius Randle should be expected to play a major role in contributing towards winning as the veteran on the team.

Out [23]:

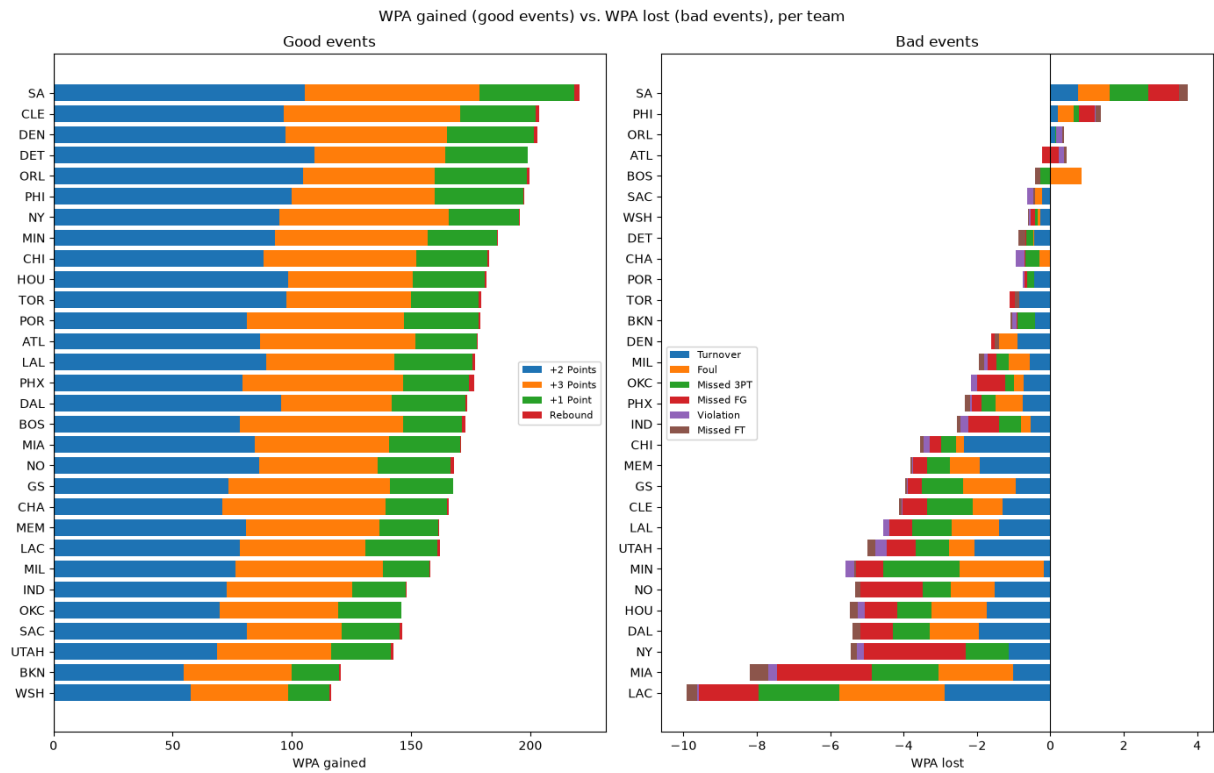
	player_name	position	wpa_total	possessions	wpa_per_100
279	Julius Randle	F	33.317001	3408	0.978
360	Michael Porter Jr.	F	18.790001	1954	0.962
152	Egor Demin	G	8.482000	1006	0.843
342	Malachi Smith	G	1.858000	224	0.830
504	Tyrese Martin	G	4.911000	604	0.813
391	Noah Clowney	F	11.791000	1591	0.741
393	Nolan Traore	G	7.717000	1073	0.719
398	Ochai Agbaji	G	5.267000	756	0.697
529	Ziaire Williams	F	7.006000	1057	0.663
472	Terance Mann	G	6.440000	1033	0.623
141	Drake Powell	G	5.399000	866	0.623
507	Tyson Etienne	G	2.028000	326	0.622
113	Day'Ron Sharpe	C	8.286000	1361	0.609
380	Nic Claxton	C	10.822000	1786	0.606
151	E.J. Liddell	F	1.808000	317	0.570
39	Ben Saraf	G	4.338000	783	0.554
107	Danny Wolf	F	6.521000	1198	0.544
271	Josh Minott	F	3.922000	727	0.539
213	Jalen Wilson	F	3.931000	739	0.532
490	Trevon Scott	F	0.444000	122	0.364
78	Chaney Johnson	F	1.062000	332	0.320

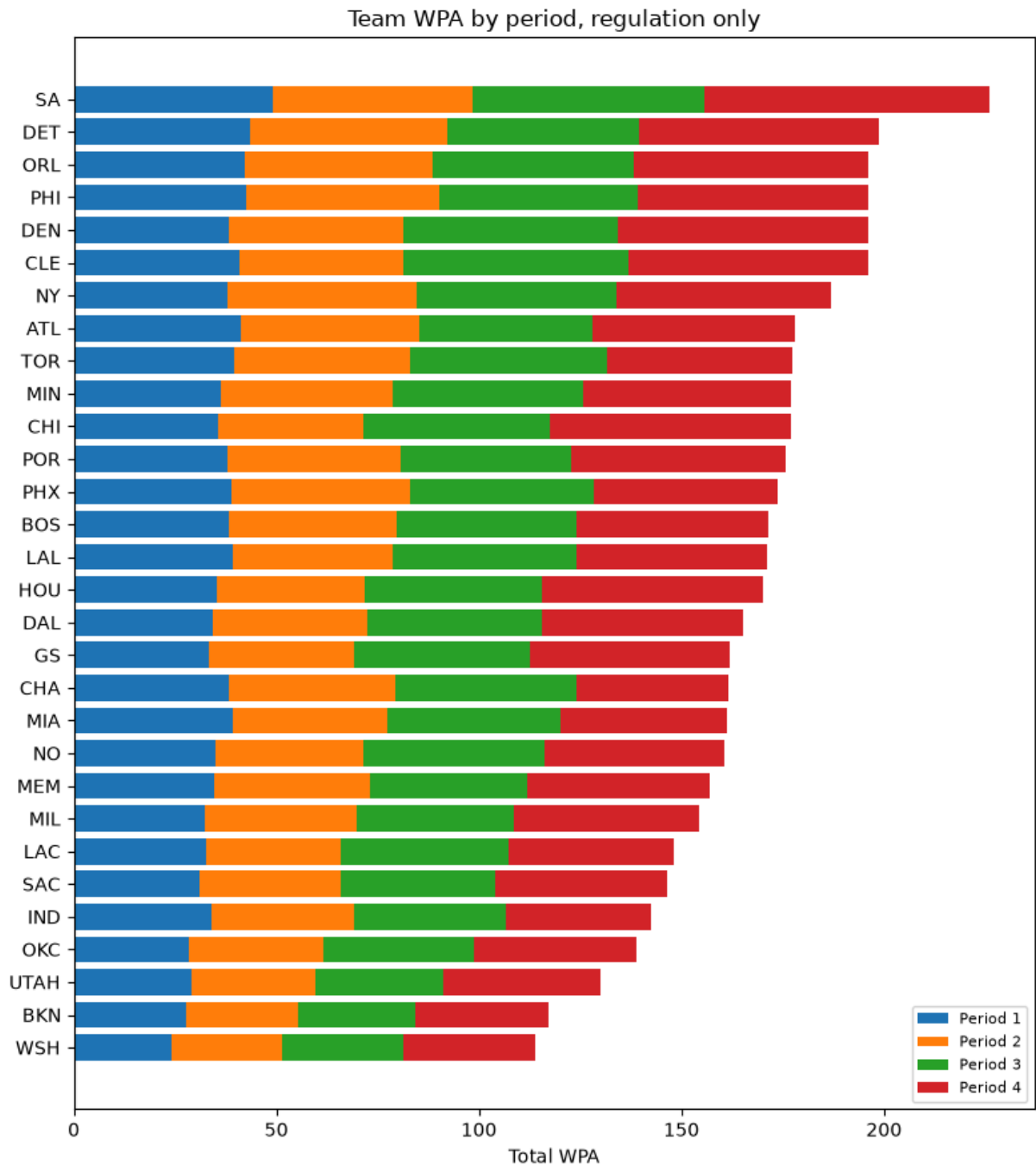
Lastly, I evaluated specific play types within the data and check which ones are most beneficial and detrimental towards winning. Sorting by average WPA in ascending order, we see bad events such as violations, turnovers, fouls, and missed shots contribute negatively towards winning on average as expected. However, their magnitude or significance in affecting WPA on a singular play-by-play basis is much lower than scoring. Shooting and scoring in today's game I believe is the single most important factor in determining winning. We have seen the NBA transform into a 3pt shooting league where teams are attempting a record number of three point shots and it's reflected in WPA calculations. This does not mean turnovers are non-significant, especially in late game situations. But stronger teams against weaker competition are able to commit turnovers early in games that don't cost the team much in terms of winning.

Out [24]:

	short_description	total_wpa	average_wpa	num_possessions
11	Violation	-1.280411	-0.000811	1578
10	Turnover	-24.719049	-0.000649	38111
3	Foul	-19.362963	-0.000360	53727
5	Missed 3PT	-17.188803	-0.000272	63150
6	Missed FG	-15.241816	-0.000247	61760
7	Missed FT	-1.115173	-0.000083	13478
8	Rebound	8.278219	0.000060	138389
4	Goaltending	0.342903	0.000523	656
9	Timeout	35.198357	0.002499	14084
0	+1 Point	847.211182	0.017525	48342
1	+2 Points	2507.610352	0.033439	74990
2	+3 Points	1720.726807	0.049510	34755

If we aggregate WPA by team and separate into good vs bad play types and sort, we can learn which teams have the highest and lowest total WPA and how much of each total team WPA value is determined by shot making, rebounding, missed shots, and turnovers. It's worth pointing out that dominant teams that have high win probabilities pregame such as OKC Thunder appear at the bottom of WPA gained. This might be due to the fact that their pregame win probabilities were already skewed towards them to begin with, which results in lower magntudinal changes to their WPA play-by-play. They also played many non-competitive 4th quarters due to blowouts which hurts their ability to accumulate more WPA. Interestingly, Spurs were at the top of the rankings and LA Clippers were the outlier team in significantly losing more WPA from turnovers, fouls, and missed shots which could help explain why they started the season 6–21. And in terms of total WPA by team grouped by quarter, we see Spurs accumulated the most WPA in 4th quarters while teams that struggled to win games had much lower WPA contributions late in games.





8. Limitations and Next Steps

This project set out to answer whether an NBA win probability model could be built from public play-by-play data, whether it would hold up against an established ESPN benchmark, and whether the resulting win probability added (WPA) metric could surface meaningful signal for player and team evaluation.

Using two full seasons (2025 training, 2026 test — roughly 1.25 million possessions total) of sportsdataverse play-by-play data, an XGBoost model built on 18 engineered features — dominated by the lagged score-differential/time-remaining interaction and pregame win probability — achieved similar test set performance to that of ESPN's win

probability models in large part due to including ESPN's pregame odds into the model. Calibration checks confirmed the model's predicted probabilities track real outcomes closely across the full range, with only slight under-prediction near 50% and in the 85-90% band.

Using this model, WPA was computed for every possession and aggregated to build player leaderboards (both total and per-100-possession), a team-level breakdown of good vs. bad event contributions, a period-by-period team profile, and individual game-level narratives (e.g., Kevin Durant's largest-impact plays in a specific game).

There are limitations in this analysis that need to be addressed:

- **Sample Size:** Two seasons is a reasonable starting point but is thin for clutch-specific reliability — a player's 4th-quarter/OT WPA ranking is based on a relatively small number of high-leverage possessions, and should be treated as provisional until validated on a longer multi-season window or checked for year-over-year persistence.
- **Defense Attribution:** Offensive WPA (shot-making, shot creation, drawing fouls) attributes cleanly to the main, acting player; defensive WPA (a defensive team stop, a forced miss, steals & blocks) is structurally harder to credit to one individual given scheme and teammate involvement, and should be read with more caution than the offensive numbers. This is why players such as Rudy Gobert or Draymond Green are undervalued in this analysis.
- **Teammate Attribution:** Teammates that set up a clean shot with their playmaking and smart offensive sets (fast breaks, transition offense, half-court offense) are ignored in this analysis. Creating a good shot is just as important as the shooter making the shot.
- **Calibration and Monotonicity:** The model shows real but modest miscalibration in specific probability bands (50%, 85-90%); isotonic calibration and monotonic constraints were deliberately left out of this version to keep the baseline simple, and would be the first thing to reintroduce if a more precise model were needed.
- **Results-Based Analysis:** Due to not having player tracking data, we are crediting WPA to plays without any context on shot quality or how a shot was created. A wide-open, uncontested shot created from an offensive set is a much better process than making a random, heavily contested heave at the end of the shot clock. However in this analysis, both situations would credit the player the same WPA.
- **Total WPA Limits:** As a cumulative metric, players logging heavy minutes and high usage rates naturally accumulate more opportunities to swing win probability positively. Missing in the creation of WPA are lack of context for shot quality and ignoring other teammates that did not attempt the shot or turnover the ball.

Next steps:

- Extend to additional seasons to build more reliable clutch-specific leaderboards and test whether player WPA rankings are actually persistent (a split-half or year-over-year correlation check) before treating them as a stable measure of skill rather than variance.
- Reintroduce isotonic calibration and monotonic constraints to address the specific probability bands where under-prediction was observed.
- Build a more rigorous possession-level (not play-level) attribution scheme, particularly for shared credit on rebounds and assists.
- Engineer more in-play features that capture scoring runs, consecutive defensive stops, shooting percentages, etc. to rely less on static pregame odds from ESPN for the model to better learn how on-court player/team performance affects win probability.